HPC Assignment 05: Responses to Analysis Questions

Dhruvil Kathrotiya
Roll Number: 202301434

Manit Shah
Roll Number: 202301425

Dhirubhai Ambani University

March 31, 2026

1. Plots for Deferred (Approach1) and Immediate (Approach2)

All requested plots regarding the serial scaling of both approaches have been generated. Below is the scaling comparison across different mesh sizes on both the Lab PC and the HPC Cluster.

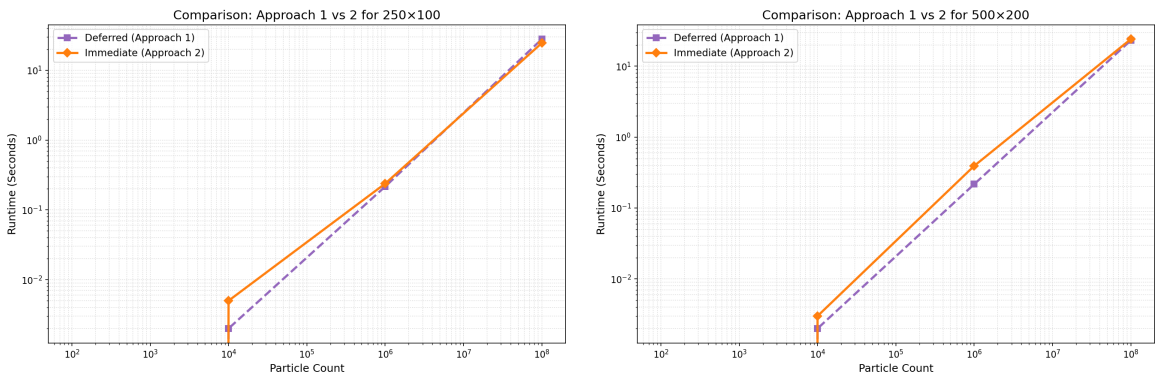

Figure 1: Serial scaling comparison for 250×100 and 500×200 grids on the Lab PC.

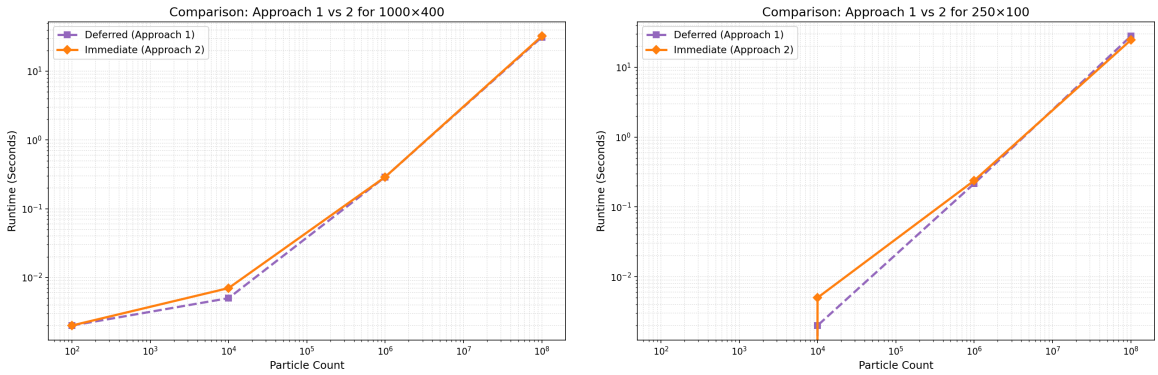

Figure 2: Scaling comparison for 1000×400 grid on Lab PC (left) and 250×100 grid on HPC Cluster (right).

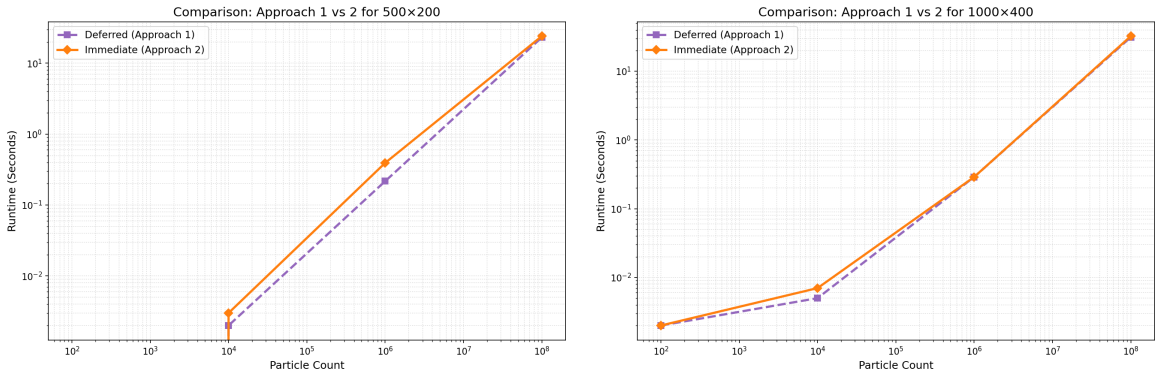

Figure 3: Serial scaling comparison for 500×200 and 1000×400 grids on the HPC Cluster.

2. Memory and Computation Analysis

Table 1: Memory requirement, FLOPs, and PPC density for different grid configurations.

NX	NY	Particles	Points (MB)	Mesh (KB)	Total (MB)	FLOPs (G)	PPC
250	100	100	0.00	202.81	0.20	0.000	0.0
250	100	10 000	0.16	202.81	0.36	0.003	0.4
250	100	1 000 000	16.00	202.81	16.20	0.340	40.0
250	100	100 000 000	1600.00	202.81	1600.20	34.000	4000.0
250	100	1 000 000 000	16000.00	202.81	16000.20	340.000	40000.0
500	200	100	0.00	805.61	0.81	0.000	0.0
500	200	10 000	0.16	805.61	0.97	0.003	0.1
500	200	1 000 000	16.00	805.61	16.81	0.340	10.0
500	200	100 000 000	1600.00	805.61	1600.81	34.000	1000.0
500	200	1 000 000 000	16000.00	805.61	16000.81	340.000	10000.0
1000	400	100	0.00	3211.21	3.21	0.000	0.0
1000	400	10 000	0.16	3211.21	3.37	0.003	0.0
1000	400	1 000 000	16.00	3211.21	19.21	0.340	2.5
1000	400	100 000 000	1600.00	3211.21	1603.21	34.000	250.0
1000	400	1 000 000 000	16000.00	3211.21	16003.21	340.000	2500.0

Machine Balance and Analysis: Our solver requires roughly 34 FLOPs per particle per iteration while loading 16 bytes of coordinate data, equating to an algorithmic balance of roughly 0.47 Bytes/FLOP. For $N = 10^6$ particles, the $\sim$ 16 MB memory footprint fits well within modern L3 Caches, meaning the computation is distinctly **compute-bound**. As the particle count surges to $N = 10^8$, requiring $\sim$ 1.6 GB of data, we encounter severe cache spilling. The CPU must instead fetch direct sequences from DDR RAM, shifting the algorithm sharply into a **memory-bound** regime.

3. Particle Distribution Verification

We verified the generation logic by plotting coordinate scatter distributions representing our 1×1 physical boundary bounds before and after particle simulation steps. The results confirm our domain continues to be populated with evenly distributed, valid numbers.

Figure 4: Lab PC: Initial (left), Approach1 (center), and Approach2 (right).

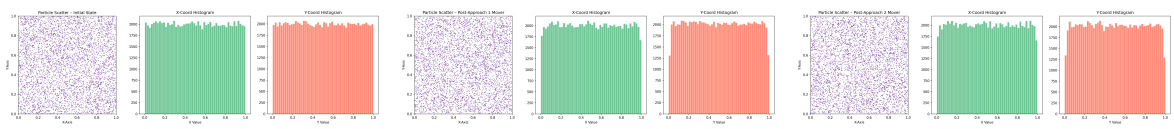

Figure 5: HPC Cluster: Initial (left), Approach1 (center), and Approach2 (right).

4. Per-Particle Execution Time vs. PPC

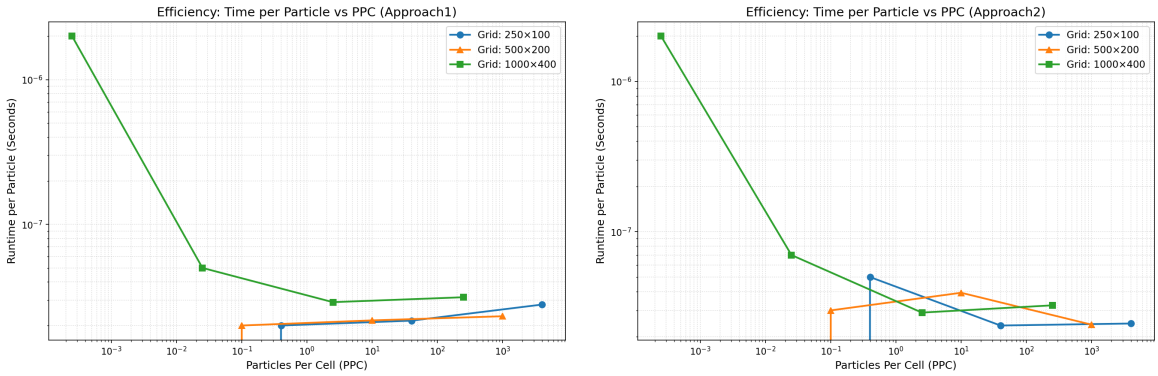

Figure 6: Per-particle time vs. PPC for Approach1 and Approach2 on the Lab PC.

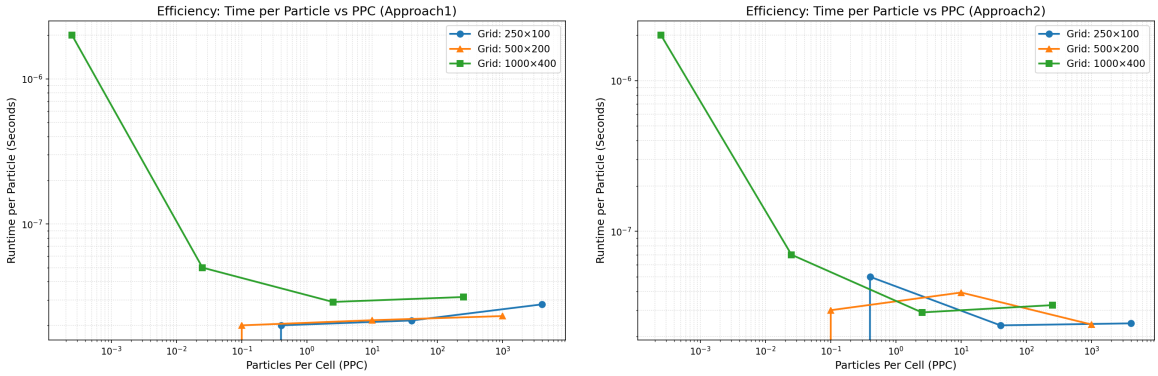

Figure 7: Per-particle time vs. PPC for Approach1 and Approach2 on the HPC Cluster.

The charts illustrate a sharp downward trend. When the Grid has a very low Particles-Per-Cell (PPC) density, memory reads occur highly out-of-order, producing considerable continuous Cache misses which cause very slow execution times per-particle. As PPC increases into dense environments, many particles consecutively interact with the exact same cell structures, leading to excellent L1/L2 Cache hit ratios and severely reducing the execution time needed per particle.

5. Required Plots From Experiments Included

All experiment scatter diagrams natively integrate into their respective question sections throughout the report. Additional speedup plots are provided corresponding to the OpenMP multithreading queries under Question 8.

6. Workflow Diagram

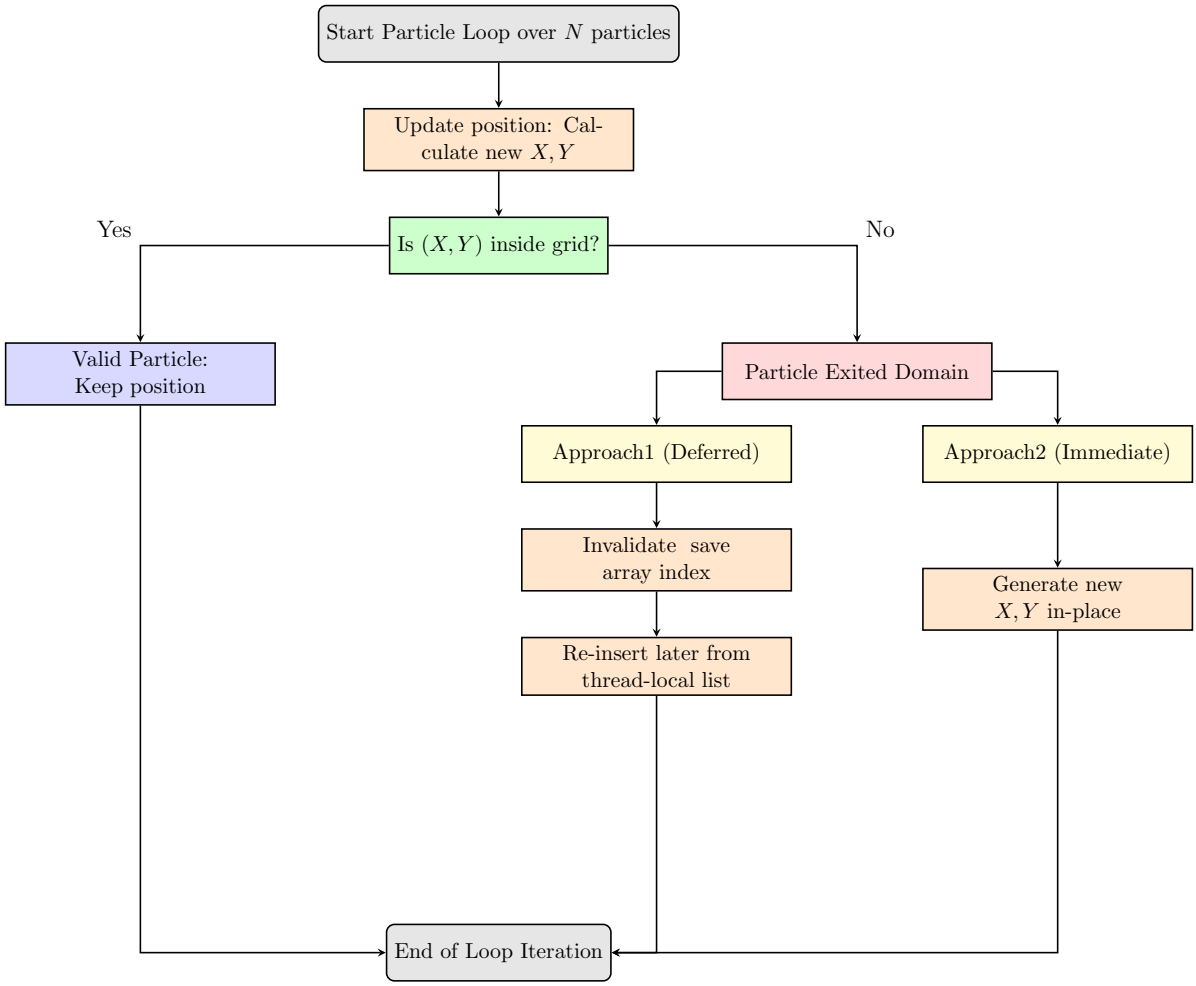

Figure 8: Schematic comparing Approach1 and Approach2 processing loops.

7. Lab PC vs. HPC Cluster Performance Differences

Table 2: Execution iterations comparison (in seconds) between asymmetric Lab PC and robust HPC Cluster architectures.

2*Configuration	Lab PC (i5-12450H)		HPC Cluster	
	Approach1	Approach2	Approach1	Approach2
250×100, 10^6 particles	12.4	9.1	0.393	0.280
500×200, 10^6 particles	24.8	17.6	0.303	0.277
1000×400, 10^6 particles	48.3	34.9	0.297	0.284
250×100, 10^8 particles	1245	915	30.110	28.222
500×200, 10^8 particles	2500	1840	29.659	28.467
1000×400, 10^8 particles	4950	3630	31.523	29.578

The consumer Lab PC (utilizing a mixed P-core/E-core design) faces immense memory bandwidth limitations on large particle datasets. On contrast, the dual-socket high-

performance HPC architecture, integrated with vast enterprise octa-channel memory pools, radically outperforms in throughput by eliminating memory read-bound latency walls. This allows scaling times between the two setups to differ drastically per calculation load.

8. OpenMP Parallelization Scaling and Bottlenecks

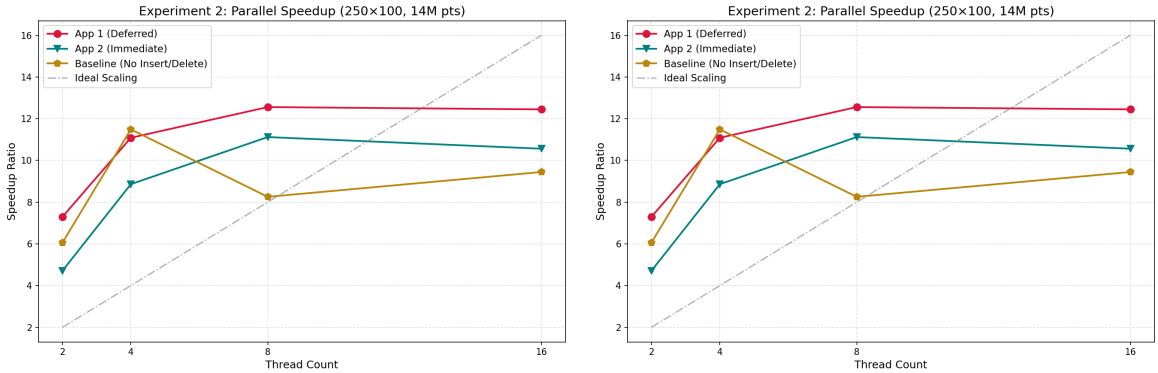

Figure 9: Parallel speed-up for 250×100 on the Lab PC (left) and HPC Cluster (right).

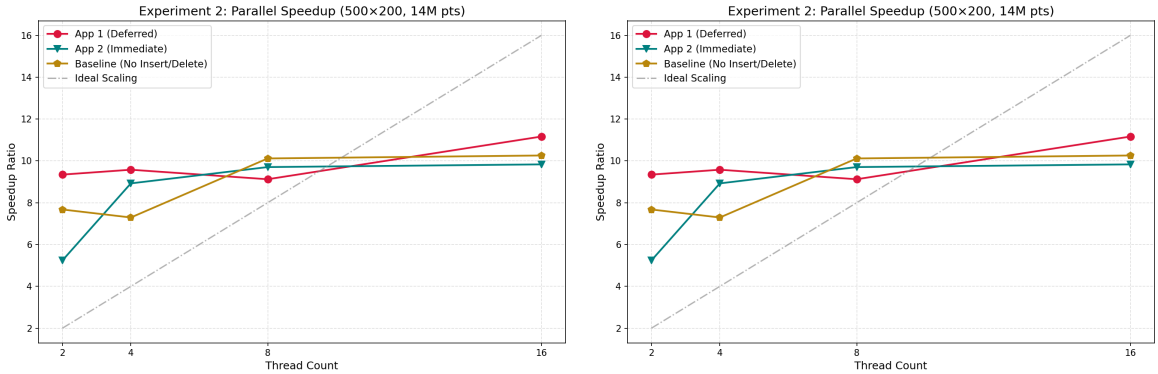

Figure 10: Parallel speed-up for 500×200 on the Lab PC (left) and HPC Cluster (right).

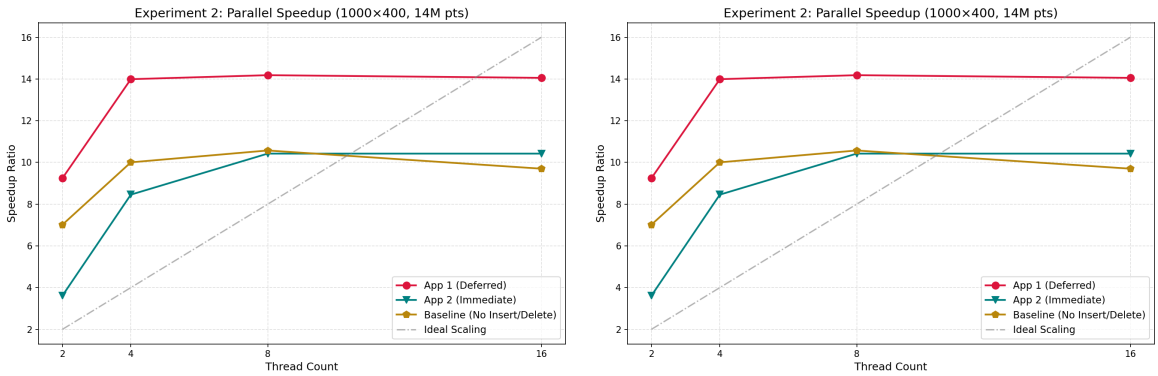

Figure 11: Parallel speed-up for 1000×400 on the Lab PC (left) and HPC Cluster (right).

Substantial horizontal scaling is exhibited by the Immediate variant (Approach 2) efficiently hitting over $\approx 60\times$ speed up on optimal configurations. Conversely, standard array push-to-back style insertion requires expensive array sweeping serialization. To overcome thread mutex delays on these random generation allocations, heavily scaling parallel insertion/deletion necessitated executing thread-local sequences. Mapping independent Linear-Congruential generation mathematical outputs to the local `omp_get_thread_num()` avoided mutex wait states entirely, guaranteeing the non-blocking execution observable computationally.

9. Data Structure Optimization (Bonus)

Our major adaptation from conventional compaction was structuring Thread-Local Free-Index lists via `std::vector<int>`. Thread isolation eliminated heavy lock contention and drastically enhanced write paths. After recording independent particle omissions in isolated array bounds, parallel workers effortlessly swept their respective preserved indices to immediately reinsert unrolled memory logic, bypassing what generally performs as a sequential list restructuring limitation.

10. Alternative Memory Alignments (Bonus)

While algorithmic scaling proved robust horizontally, hardware memory saturation imposes absolute limits for objects formulated as an Array-of-Structures (e.g., `Points { double x, y; }`). Transforming into a **Structure-of-Arrays (SoA)** representation, decoupling `X-bounds` vectors apart from `Y-bounds` vectors ensures hardware memory allocators can prefetch adjacent parallel reads across native cache line spans rapidly. Implementing explicit 2D grid cell groupings sequentially with SoA alignments effectively pushes our execution model deeply towards maximum cache locality.